

ユニット 8・第13-14週

ネットワーク：ソケット

📅 第13-14週・日程は未定 📄 演習 TD7（解答 + 生成AIチャレンジ） 🎬 アニメーション 2 本

マニュアル M8・ユニットの理論

ネットワーク：ソケット

ユニット 8 の基本の読み物（約1時間 30 分）。出力はすべて実物で、コースのマシンの 127.0.0.1 上で TD7 のプログラムを実行したものです。

目次

- | | |
|----------------------|------------------------|
| §1 ソケットはディスクリプタ | §4 サーバーのモデル |
| §2 TCP 接続のライフサイクル | §5 データグラムと今日のネットワーク |
| §3 アドレス、ポート、名前 | |

§1 ソケットはディスクリプタ

ソケットは通信の端点です。 `socket(domain, type, protocol)` は `open` や `pipe` と同じくファイルディスクリプタを返します：`read`、`write`、`close` で使い、`fork` で受け継がれ、ディスクリプタ表の1項目を占めます。ドメインは使うアドレスの種類を表します：`AF_UNIX`（同じマシン上の、ファイルシステムのパス）、`AF_INET`（IPv4）、`AF_INET6`（IPv6）。タイプは通信の仕方を表します：

- **接続型**（`SOCK_STREAM`、インターネットでは TCP）：接続を確立し、その後パイプのように、信頼できて順序が保たれた**バイトストリーム**が流れます。スライド 181 の電話です。
- **非接続型**（`SOCK_DGRAM`、UDP）：送信のたびに独立した**データグラム**になり、失われたり、重複したり、順不同で届いたりします。電報です。

違いは測れます。TCP で 3 回 `write` したものが 1 回の `read` で届きます（TCP はメッセージについて何も知りません）。一方 UDP では `recvfrom` のたびにデータグラムが丸ごと返ります。そして受け手が間に合うように読まないと、UDP は同じマシンの上でさえ、誰にもエラーを返さずにデータを失います：

実際の出力

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom
#3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992
lost
      kernel counter Udp RcvbufErrors grew by 9992
```

実的な帰結：TCP では各メッセージの終わりをアプリケーションが決めます（HTTP/1 は行と Content-Length、TD7 のエコーサーバは行末）。またパイプと同じく、`read` と `write` は部分的になりえます（ユニット 2）：`write_all` はすべて送るまでループします。

実践 → A8.1 のシーン 2 と 6・TD7・4

§2 TCP 接続のライフサイクル

サーバはソケットを作り、`bind` でアドレスとポートを与え、`listen(fd, backlog)` を呼びます：ソケットは受動的（LISTEN 状態）になり、接続を受けるために使われます。クライアントは自分のソケットを作り、サーバのアドレスを指定して `connect` を呼びます。すると 2 つのカーネルが **3 ウェイハンドシェイク**を行います：クライアントから SYN、サーバから SYN-ACK、クライアントから ACK。要点は、**ハンドシェイクを完了させるのはカーネル**であって、アプリケーションではないことです。接続は待ち受けソケットの `accept` キューに ESTABLISHED のまま置かれ、`accept` はそれを取り出して、そのクライアント用の**新しいディスクリプタ**を返すだけです（待ち受け用は他の接続を受け付け続けます）。

`tcp_states` プログラムは 1 つのプロセスで両方の役を演じ、`getsockopt(TCP_INFO)` で各ソケットの状態をカーネルに尋ねます：

実際の出力

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server
ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server
CLOSE_WAIT
server read                | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server
CLOSED

                          | the client's end moved to a TIME_WAIT entry of the kernel
(60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0        0      127.0.0.1:42772   127.0.0.1:PORT
```

クローズは対称的で、半分ずつ行われます。先に `close`（または `shutdown(fd, SHUT_WR)`）を呼んだ側が FIN を送り、FIN_WAIT1 と FIN_WAIT2 を通ります。相手側は FIN を受け取って CLOSE_WAIT になり、その `read` は 0 を返します（書き手のいないパイプと同じファイル終端）。相手側も閉じて（LAST_ACK）最後の ACK が届くと、先に閉じた側は最後の ACK が失われて FIN が繰り返される場合に備え、2-MSL（Linux では 60 秒）の間 **TIME_WAIT** に残ります。この項目はプログラムより長く残り（`ss` で見えます）、これが `SO_REUSEADDR` の理由です：これがないと、サーバの再起動が `EADDRINUSE` で失敗します。

backlog はサーバが処理するクライアントの数ではなく、**accept** を待てる確立済みの接続の数を制限します。Linux は backlog + 1 個まで受け入れ、キューが満杯だと SYN を捨て、クライアントは指数的に間隔を空けて再送します（1 秒、2 秒、4 秒...）：

実際の出力

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:          accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:      accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

🔗 興味のある人へ：SYN クッキー

backlog がとても小さいと（`./backlog 2 8`）、カーネルはやがて SYN クッキーで応答します。状態を持たない、SYN フラッドへの防御です。accept キューが満杯のときに最後の ACK が届くとそれは失われ、クライアントは接続したつもりでも、サーバはそれを知りません。TD7・4 では `nstat` のカウンタがそれを示します。だから実際のサーバは大きな backlog を求めます（上限は `net.core.somaxconn`）。

実践 → A8.1 接続のライフサイクル・TD7・1 と 4

§3 アドレス、ポート、名前

ドメインごとにアドレス構造体があります（最大 108 バイトのパスを持つ `sockaddr_un`、IPv4 アドレスとポートを持つ `sockaddr_in`、IPv6 用の `sockaddr_in6`）。どれもサイズとともに `struct sockaddr *` として `bind`、`connect`、`accept` に渡し、`struct sockaddr_storage` はそのどれでも入る大きさです。構造体の中の数値は**ネットワークバイト順**（ビッグエンディアン）です：だからポートには `htons`、アドレスには `htonl` を使います。スライド 187 の例はそれを忘れており、バグが見えます：

実際の出力

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110, sockaddr_storage 128
```

x86-64 では 7000 = 0x1B58 は 58 1B というバイト列で保存されます。ビッグエンディアンとして読むと 0x581B = 22555 になり、サーバは別のポートで待ち受けることになります。同じ出力はさらに 3 つのことを示します：`getaddrinfo` は名前とサービス（`/etc/hosts` と `/etc/services`、または DNS から）を、そのまま使える IPv4 と IPv6 のアドレスの一覧に変えること（IPv4 しか知らず、スレッドセーフでなく、2008 年に POSIX から外れた `gethostbyname` の代わり）、1024 未満のポートには特権（`CAP_NET_BIND_SERVICE`）が必要なこと、そして構造体の実際のサイズです。

`AF_UNIX` ソケットは IP もポートも使いません：アドレスはソケット型のファイルです。さらにカーネルは、相手側が誰かをサーバに教えます（`SO_PEERCRED`：PID と UID）。`systemd`、`D-Bus`、`/var/run/docker.sock` はこれを使って何を許すかを決めます：

実際の出力

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108
bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

実践 → TD7・4 のパート 5 と 6

§4 サーバーのモデル

サーバはほとんどの時間を待つことに費やします：クライアントが来るのを待つ（`accept`）、またはそのクライアントが何かを送るのを待つ（`read`）。その待ち方の組み立てで、誰が誰を待つかが決まります：

- **反復型**：1つのスレッドが一度に1つのクライアントを処理する。あるクライアントが遅いと、他は全員キューで待つ。
- **クライアントごとのプロセス**（`accept` の後に `fork`）：子が処理し、親は `accept` に戻る。互いに隔離されるが、プロセスの作成はスレッドの何倍もかかり（TD6・5 で測定）、ゾンビを避ける必要がある。
- **クライアントごとのスレッド**：安価でメモリも共有できるが、10,000 の接続はスタック付きの 10,000 スレッドになる。
- **スレッドプール**：起動時に作った k 個のスレッドが、メインスレッドの満たす有限のキューから接続を取り出す（ユニット 7 の生産者-消費者）。資源は限られるが、遅いクライアントが k 個来ると飽和する。
- **イベントループ**：1つのスレッドが `epoll_wait` でカーネルにどのソケットにもうデータがあるかを尋ね、それだけを処理するので、遅いクライアントでブロックすることはない。C10K 問題への答えで、`nginx` や `Node.js` のモデル。`io_uring`（Linux 5.1）はこの考えをさらに進めた。

TD7 は 5 つすべてを `http_server.c` に実装し、32 のクライアントで測ります。まず 1 つ、次に 8 つのクライアントが要求を送るのに 1 秒かかります：

実際の出力

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 982 ms, max 983 ms
```

実際の出力

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 981 ms, max 981 ms
```

実際の出力

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 1 ms, max 2 ms
```

実践 → A8.2 サーバモデル・TD7・3 と 4

§5 データグラムと今日のネットワーク

非接続モードには `listen` も `accept` もありません：サーバはよく知られたポートを `bind` し、`recvfrom` で待ちます。`recvfrom` はデータグラムと送り手のアドレスを返します。クライアントは宛先アドレスを指定して `sendto` を使います（スライド 194～197）。UDP ソケットを `connect` して宛先を固定し、`send` / `recv` を使うこともできます。UDP は DNS、リアルタイムの音声と映像、そして 2021 年からは **QUIC** (RFC 9000) の土台で、その上で **HTTP/3** (RFC 9114) が動きます：信頼性と暗号化はその上に、カーネルではなくプログラムの中で実装されます。

元のコースとの違いがさらに 2 つあります。ほぼすべての通信は **TLS** (https、443) で暗号化されています：ソケットは暗号化されたバイトを運び、ライブラリ (OpenSSL、GnuTLS...) が

処理します。そしてスライド 183 の暗号化なしのサービス（telnet、ftp）は、ssh と https に置き換えました。

実践 → A8.1 のシーン 6（UDP）・TD7・4